Example: the MouseMotionListener interface defines two methods to receive notifications when the mouse is dragged or moved.

10.3 EVENT CLASSES

The classes that represent events are responsible for Java's event handling mechanism.

EventObject class is in java.util. It is placed at the root of the Java event class hierarchy. EventObject is the superclass of all events. AWTEvent is a superclass of all AWT events that are handled by the delegation event model.

EventObject contains two methods: getSource( ) & toString( ).

The getSource( ) method returns the source of the event. Its general form is : Object getSource( )

The toString( ) returns the string equivalent of the event.

The package java.awt.event defines many types of events that are generated by various user interface elements.

Event Class	Description
ActionEvent	Generated when a **button is pressed**, a **list item is double-clicked**, or a **menu item is selected**.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a **check box or list item is clicked**; also occurs when a **choice selection is made** or a **checkable menu item is selected or deselected.**
KeyEvent	Generated when input is received from

	the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when the mouse wheel is moved.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

10.3.1. The ActionEvent Class

An ActionEvent is generated when a button is pressed, a list item is double-clicked, or a menu item is selected.

The ActionEvent class defines four integer constants that can be used to identify any modifiers associated with an action event:

ALT_MASK, CTRL_MASK, META_MASK, and SHIFT_MASK.

There is an integer constant, ACTION_ PERFORMED, which can be used to identify action events.

ActionEvent has these three constructors:

ActionEvent(Object src, int type, String cmd)

ActionEvent(Object src, int type, String cmd, int modifiers)

ActionEvent(Object src, int type, String cmd, long when, int modifiers)

Here, src is a reference to the object that generated this event. The type of the event is specified by type, and its command string is cmd. The argument modifiers indicates which modifier keys (ALT, CTRL, META, and/or SHIFT) were pressed when the event was generated. The when parameter specifies when the event occurred.

When a button is pressed, an action event is created. The getActionCommand() method helps to obtain the command name of an ActionEvent. The command name is usually the **text written on the button**. For example, if you press a button labeled "**Login**", getActionCommand() will return "Login".

Its form is:

String getActionCommand( )

For example, If we click a button with the label "**Login**", getActionCommand() will return Login.

The getModifiers( ) method returns a value that indicates which modifier keys (ALT, CTRL, META, and/or SHIFT) were pressed when the event was generated.

Its form is:

int getModifiers( )

The method getWhen( ) returns the time at which the event took place. This is called the event's timestamp.

Its form is::

long getWhen( )

10.3.2. The AdjustmentEvent Class

An AdjustmentEvent is generated by a scroll bar. There are five types of adjustment events.

The AdjustmentEvent class defines integer constants to identify them.

BLOCK_DECREMENT The user clicked inside the scroll bar to decrease its value.

BLOCK_INCREMENT The user clicked inside the scroll bar to increase its value.

TRACK The slider was dragged.

UNIT_DECREMENT The button at the end of the scroll bar was clicked to decrease its value.

UNIT_INCREMENT The button at the end of the scroll bar was clicked to increase its value.

ADJUSTMENT_VALUE_CHANGED indicates that a change has occurred.

AdjustmentEvent constructor is:

AdjustmentEvent(Adjustable src, int id, int type, int data)

Here, src is a reference to the object that generated this event. The id specifies the event. The type of the adjustment is specified by type, and its associated data is data. The getAdjustable( ) method returns the object that generated the event.

Its form is:

Adjustable getAdjustable( )

getAdjustmentType( ) method-gives details about the type of the adjustment event.

The general form is :

int getAdjustmentType( )

getValue( ) method- To retrieve the amount of the adjustment:

int getValue( )

10.3.3 The ComponentEvent Class

A ComponentEvent is generated when the size, position, or visibility of a component is changed. There are four types of component events.

The ComponentEvent class defines integer constants that can be used to identify them.

COMPONENT_HIDDEN The component was hidden.
COMPONENT_MOVED The component was moved.
COMPONENT_RESIZED The component was resized.
COMPONENT_SHOWN The component became visible.

Constructor of ComponentEvent is:

ComponentEvent(Component src, int type)

Here, src is a reference to the object that generated this event. The type of the event is specified by type.

The getComponent( ) method returns the component that generated the event.

Its form is:

Component getComponent( )

10.3.4. The ContainerEvent Class

A ContainerEvent is generated when a component is added to or removed from a container.

There are two types of container events. The ContainerEvent class defines int constants that can be used to identify them: COMPONENT_ADDED and COMPONENT_REMOVED.

Constructor is:

ContainerEvent(Component src, int type, Component comp)

Here, src is a reference to the container that generated this event. The type of the event is specified by type, and the component that has been added to or removed from the container is comp.

getContainer( ) method - to obtain a reference to the container that generated this event.

Container getContainer( )

The getChild( ) method - returns a reference to the component that was added to or removed from the container. Its general form is:

Component getChild( )

10.3.5 The FocusEvent Class

A FocusEvent is generated when a component gains or loses input focus. These events are identified by the integer constants - FOCUS_GAINED, FOCUS_LOST

FocusEvent is a subclass of ComponentEvent and has these constructors:

FocusEvent(Component src, int type)

FocusEvent(Component src, int type, boolean temporaryFlag)

FocusEvent(Component src, int type, boolean temporaryFlag, Component *other*)

Here, src is a reference to the component that generated this event. The type of the event is specified by type.

The argument temporaryFlag is set to true if the focus event is temporary. Otherwise, it is set to false.

If a FOCUS_GAINED event occurred, *other in constructor* will refer to the component that lost focus. Conversely, if a FOCUS_LOST event occurred, *other* will refer to the component that gains focus.

We can determine the *other* component by calling getOppositeComponent( ):

Component getOppositeComponent( )

The isTemporary( ) method indicates if this focus change is temporary. Its form is e:

boolean isTemporary( )

The method returns true if the change is temporary.

10.3.6. The InputEvent Class

The abstract class InputEvent is a subclass of ComponentEvent and is the superclass for component input events. Its subclasses are KeyEvent and MouseEvent.

InputEvent class defined the following eight values to represent the modifiers:

ALT_MASK BUTTON2_MASK
META_MASK

ALT_GRAPH_MASK BUTTON3_MASK
SHIFT_MASK

BUTTON1_MASK CTRL_MASK

Extended modifier values are also there:

ALT_DOWN_MASK BUTTON2_DOWN_MASK
META_DOWN_MASK

ALT_GRAPH_DOWN_MASK BUTTON3_DOWN_MASK
SHIFT_DOWN_MASK

BUTTON1_DOWN_MASK CTRL_DOWN_MASK

isAltDown( ), isAltGraphDown( ), isControlDown( ), isMetaDown( ), and isShiftDown( ) methods are to test if a modifier was pressed at the time an event is generated.

To obtain a value that contains all of the original modifier flags call **getModifiers( ) method:**

int getModifiers( )

To obtain the extended modifiers call **getModifiersEx( ):**

int getModifiersEx( )

10.3.7 The ItemEvent Class

An ItemEvent is generated when a check box or a list item is clicked or when a checkable menu item is selected or deselected.

There are two types of item events, which are identified by the following integer constants:

DESELECTED The user deselected an item.

SELECTED The user selected an item.

ItemEvent defines one integer constant, ITEM_STATE_CHANGED, that signifies a change of state.

Constructor is:

ItemEvent(ItemSelectable src, int type, Object entry, int state)

src is a reference to the component that generated this event. For example, this might The type of the event is specified by type. The current state of that item is in state.

The getItem( ) method - to obtain a reference to the item that generated an event.

Its signature is:

Object getItem( )

The getItemSelectable( ) method can be used to obtain a reference to the ItemSelectable

object that generated an event. Its general form is:

ItemSelectable getItemSelectable( )

The getStateChange( ) method returns the state change (that is, SELECTED or DESELECTED)

int getStateChange( )

10.3.8. The KeyEvent Class

A KeyEvent is generated when keyboard input occurs.

There are three types of key events, which are identified by these integer constants: KEY_PRESSED, KEY_RELEASED, and KEY_TYPED.

There are many other integer constants

VK_ALT VK_DOWN VK_LEFT VK_RIGHT

VK_CANCEL VK_ENTER VK_PAGE_DOWN VK_SHIFT

VK_CONTROL VK_ESCAPE VK_PAGE_UP VK_UP

The VK constants specify virtual key codes. VK_0 through VK_9 and VK_A through VK_Z define the *ASCII equivalents of the numbers and letters.*

KeyEvent is a subclass of InputEvent. One of its constructors:

KeyEvent(Component src, int type, long when, int modifiers, int code, char ch)

Here, src is a reference to the component that generated the event. The type of the event is specified by type. The system time at which the key was pressed is passed in when. The modifiers argument indicates which modifiers were pressed when this key event occurred. The virtual key code, such as VK_UP, VK_A, and so forth, is passed in code.

getKeyChar( ) method- returns the character that was entered,

getKeyCode( ) method returns the key code. Their general forms are

char getKeyChar( )

int getKeyCode( )

If no valid character is available, then getKeyChar( ) returns CHAR_UNDEFINED. When a KEY_TYPED event occurs, getKeyCode( ) returns VK_UNDEFINED.

10.3.9. The MouseEvent Class

There are eight types of mouse events. The MouseEvent class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED	The user clicked the mouse.
MOUSE_DRAGGED	The user dragged the mouse.
MOUSE_ENTERED	The mouse entered a component.
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved.
MOUSE_PRESSED	The mouse was pressed.
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved

MouseEvent is a subclass of InputEvent. Here is one of its constructors:

MouseEvent(Component src, int type, long when, int modifiers,

int x, int y, int clicks, boolean triggersPopup)

Here, src is a reference to the component that generated the event. The type of the event is specified by type. The system time at which the mouse event occurred is passed in when. The modifiers argument indicates which modifiers were pressed when a mouse event occurred. The coordinates of the mouse are passed in x and y. The click count is passed in clicks. The triggersPopup flag indicates if this event causes a pop-up menu to appear on this platform.

The X and Y coordinates of the mouse within the component when the event occurred can be obtained using getX() and getY( ) respectively;

```
int getX( )

int getY( )
```

getPoint( ) method - to obtain the coordinates of the mouse.

```
Point getPoint( )
```

The translatePoint( ) method- changes the location of the event.

Its form is:

```
void translatePoint(int x, int y)
```

The getClickCount( ) method - obtains the number of mouse clicks for this event.

Its signature is:

```
int getClickCount( )
```

The isPopupTrigger( ) method - tests if this event causes a pop-up menu to appear on this platform. Its form is:

```
boolean isPopupTrigger( )
```

int getButton( ) - It returns a value that represents the button that caused the event. The return value will be one of these constants defined by MouseEvent:

NOBUTTON BUTTON1 BUTTON2 BUTTON3

The NOBUTTON value indicates that no button was pressed or released.

Methods to obtain the coordinates of the mouse relative to the screen rather than the component are::

```
Point getLocationOnScreen( )

int getXOnScreen( )

int getYOnScreen()
```

10.3.10. The MouseWheelEvent Class

The MouseWheelEvent class encapsulates a mouse wheel event. It is a subclass of MouseEvent.

Not all mice have wheels.

Mouse wheels are used for scrolling. MouseWheelEvent defines these two integer constants:

WHEEL_BLOCK_SCROLL — A page-up or page-down scroll event occurred.

WHEEL_UNIT_SCROLL — A line-up or line-down scroll event occurred.

One of the constructors defined by MouseWheelEvent:

MouseWheelEvent(Component src, int type, long when, int modifiers,

int x, int y, int clicks, boolean triggersPopup, int scrollHow, int amount, int count)

Here, src is a reference to the object that generated the event. The type of the event is specified by type. The system time at which the mouse event occurred is passed in when. The modifiers argument indicates which modifiers were pressed when the event occurred. The coordinates of the mouse are passed in x and y. The number of clicks the wheel has rotated is passed in clicks. The triggersPopup flag indicates if this event causes a pop-up menu to appear on this platform. The scrollHow value must be either WHEEL_UNIT_SCROLL or WHEEL_BLOCK_SCROLL. The number of units to scroll is passed in amount. The count parameter indicates the number of rotational units that the wheel moved.

MouseWheelEvent defines methods that give you access to the wheel event

int getWheelRotation( ) – It returns the number of rotational units. If the value is positive, the wheel moved counterclockwise. If the value is negative, the wheel moved clockwise.

int getScrollType( ) –To obtain the type of scroll.

It returns either WHEEL_UNIT_SCROLL or WHEEL_BLOCK_SCROLL.

If the scroll type is WHEEL_UNIT_SCROLL, then we can call

int getScrollAmount( ) - to obtain the number of units to scroll.

10.3.11 The TextEvent Class

Instances of TextEvent class describe text events. These are generated by text fields and text areas when characters are entered by a user or program.

TextEvent defines the integer constant TEXT_VALUE_CHANGED.

One constructor is:

TextEvent(Object src, int type)

Here, src is a reference to the object that generated this event. The type of the event is specified by type.

10.3.12. The WindowEvent Class

TheWindowEvent class defines integer constants that can be used to identify ten window events.. The constants and their meanings are:

WINDOW_ACTIVATED The window was activated.

WINDOW_CLOSED The window has been closed.

WINDOW_CLOSING The user requested that the window be closed.

WINDOW_DEACTIVATED The window was deactivated.

WINDOW_DEICONIFIED The window was deiconified.

WINDOW_GAINED_FOCUS The window gained input focus.

WINDOW_ICONIFIED The window was iconified.

WINDOW_LOST_FOCUS The window lost input focus.

WINDOW_OPENED The window was opened.

WINDOW_STATE_CHANGED The state of the window changed

The constructors are:

WindowEvent(Window src, int type)

WindowEvent(Window src, int type, Window other)

WindowEvent(Window src, int type, int fromState, int toState)

WindowEvent(Window src, int type, Window other, int fromState, int toState)

Here, other specifies the opposite window when a focus or activation event occurs. The fromState specifies the prior state of the window, and toState specifies the new state that the window will have when a window state change occurs.

A commonly used method in this class is getWindow( ).

Window getWindow( ) - returns the Window object that generated the event.

Window getOppositeWindow( ) - return the opposite window (when a focus or activation event has occurred)

int getOldState( ) - returns the previous window state,

int getNewState( ) returns the current window state.

10.4 SOURCES OF EVENTS

The components in the user interface that can generate the events are the **sources of the event.**

Event Source	Description of event generated	
Button	Generates action events when the button is pressed.	ActionEvent
Check box	Generates item events when the check box is selected or deselected.	ItemEvent
Choice	Generates item events when the choice is changed	ItemEvent
List	Generates action events when an item is double- clicked; generates item events when an item is selected or deselected.	ActionEvent ItemEvent